

GroundTruth RAG vs Direct

Protótipo para comparar eficiência e confiabilidade em arquiteturas baseadas em modelos de linguagem

Versão documentada em 13 de setembro de 2026

O que eu finalizei

Finalizei um protótipo executável que aplica a mesma pergunta a duas abordagens: geração direta e geração com recuperação de contexto. O sistema reúne as respostas, as fontes recuperadas e as principais medidas operacionais em uma única tela. Ele também mantém um histórico local para que eu possa rever e comparar experimentos anteriores.

A entrega já inclui uma interface web, uma API, um mecanismo local de recuperação, integração opcional com a API da OpenAI e testes automatizados. O modo demonstrativo funciona sem credenciais. Isso permite apresentar o fluxo completo antes de conectar modelos pagos ou documentos oficiais.

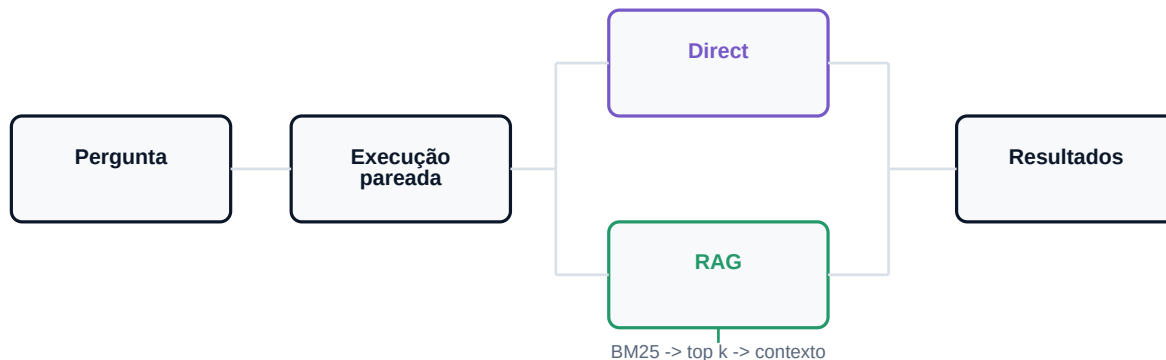
Minha conclusão neste estágio

O protótipo é suficiente para demonstrar a hipótese e organizar um estudo piloto. Ele ainda não produz evidência científica por conta própria. Para isso, preciso substituir o corpus demonstrativo, preparar perguntas anotadas e repetir as execuções com um protocolo de avaliação definido antes da coleta.

Situação atual	O que já funciona
Aplicação	Interface responsiva, API e build de produção
Comparação	Execução Direct e RAG com os mesmos parâmetros
Recuperação	BM25 local, chunking configurável e seleção top k
Medição	Latência, tokens, custo estimado, cobertura, aderência e citações
Validação	Cinco testes aprovados, verificação de tipos e auditoria de dependências

Como o protótipo foi organizado

A aplicação foi dividida em uma camada de interface, uma API de experimento e módulos isolados para corpus, recuperação, geração, avaliação e persistência. Essa separação me permite trocar o modelo ou o mecanismo de busca sem reescrever o restante do sistema.



Camada de interface

A interface em React concentra a configuração do experimento e a leitura dos resultados. Nela eu escolho a pergunta, o provedor, o modelo, o número de trechos recuperados e o tamanho dos chunks. A tela apresenta as respostas lado a lado, destaca as métricas e mostra o texto das evidências recuperadas.

API e orquestração

O servidor Express recebe a configuração, valida os limites e executa Direct e RAG em paralelo. A execução paralela evita que um método seja favorecido apenas por ter sido chamado primeiro. Cada resultado registra a resposta, o modelo efetivamente usado, a latência, a contagem de tokens e as métricas de qualidade.

Recuperação

O corpus é dividido por sentenças até atingir o tamanho aproximado de chunk definido na interface. A busca usa BM25, um método lexical que pondera a frequência do termo, sua raridade no corpus e o comprimento do documento. Ele é simples, rápido e reproduzível para um corpus pequeno. Não há banco vetorial nesta versão.

Geração

No caminho Direct, o modelo recebe apenas a pergunta e uma instrução geral. No caminho RAG, ele também recebe os trechos recuperados, numerados como fontes. A instrução pede que a resposta use somente esse contexto, cite as fontes e informe quando o corpus não contiver a resposta.

Fluxo de uma execução

Cada execução segue o mesmo percurso e gera um registro identificável. O histórico guarda as últimas cem execuções em um arquivo JSON local.

Etapa	Direct	RAG
1 Pergunta	Recebe a pergunta	Recebe a mesma pergunta
2 Recuperação	Não consulta o corpus	BM25 seleciona os top k chunks
3 Prompt	Instrução geral	Instrução + trechos numerados
4 Geração	Conhecimento paramétrico	Resposta limitada ao contexto
5 Medição	Tokens, latência e qualidade	Tokens, latência, qualidade e citações
6 Registro	Persistido no mesmo experimento	Persistido no mesmo experimento

Componentes principais

Arquivo	Responsabilidade
server/index.ts	Rotas, validação, execução pareada, custo e agregados
server/rag.ts	Tokenização, chunking e ranking BM25
server/llm.ts	Modo demo e cliente da OpenAI Responses API
server/evaluation.ts	Métricas heurísticas de qualidade
server/store.ts	Persistência local do histórico
server/corpus.ts	Fontes demonstrativas
src/App.tsx	Configuração, comparação, histórico e visualização do corpus

Decisões que mantive simples

Usei persistência em JSON porque o volume atual é pequeno e a prioridade era tornar o experimento visível. Também mantive a recuperação lexical para ter um baseline interpretável. Em uma etapa posterior, posso comparar BM25, embeddings e busca híbrida sem alterar o contrato das respostas.

Como executar e usar

O projeto requer Node.js 22 ou superior. A instalação prepara a interface e o servidor no mesmo workspace.

```
npm install
npm run dev
```

Durante o desenvolvimento, a interface fica disponível em **http://localhost:5173** e a API em **http://localhost:8787**. O comando **npm start** serve o build já gerado pela API. Para gerar novamente a versão de produção, uso **npm run build**.

Uso sem chave de API

O motor Demo determinístico permite testar a navegação, a recuperação e o registro das métricas sem custo externo. Ele não representa a qualidade de um LLM real e não deve entrar nos resultados do estudo. Sua função é validar o encadeamento técnico e facilitar apresentações.

Uso com a OpenAI

Defino a chave no ambiente antes de iniciar o servidor. A chave não deve ser adicionada ao repositório nem escrita no navegador.

```
$env:OPENAI_API_KEY="sua-chave"
$env:OPENAI_MODEL="gpt-4.1-mini"
npm run dev
```

Quando a API detecta a variável, a interface libera o provedor OpenAI. O servidor envia as duas chamadas ao endpoint Responses e usa os números de tokens retornados pela própria API quando estão disponíveis.

Configurações do experimento

Parâmetro	Efeito	Orientação inicial
Modelo	Altera qualidade, custo e latência	Fixar um modelo por rodada
Top k	Define quantos chunks entram no contexto	Começar com 3 e testar 1, 3 e 5
Chunk	Controla a granularidade das evidências	Testar entre 300 e 800 caracteres
Temperatura	Afeta variação da resposta	Usar 0 no benchmark principal
Gabarito	Permite estimar cobertura	Preencher com resposta revisada

Como usar dados reais

O corpus atual contém resumos autorais sobre temas da BNCC. Ele foi criado para demonstrar o software e não deve ser apresentado como transcrição oficial. Para usar dados reais, eu começaria pela definição do recorte documental, não pelo upload indiscriminado de arquivos.

Preparação do corpus

- Selecionar documentos oficiais e registrar título, órgão responsável, edição, data de acesso e endereço de origem.
- Manter o arquivo original em uma pasta somente de leitura e gerar uma versão normalizada para indexação.
- Extrair o texto preservando títulos, capítulos, seções, páginas e outros elementos que permitam localizar a evidência.
- Remover cabeçalhos repetidos, rodapés, sumários duplicados e erros de OCR antes de criar os chunks.
- Atribuir um identificador estável a cada documento e a cada trecho para que uma execução possa ser reproduzida.

Formato de dados recomendado

Na versão atual, cada fonte possui quatro campos: identificador, título, seção e texto. Para uma coleta real, eu ampliaria o esquema antes da ingestão.

Campo	Uso
id	Identificador estável do documento
title	Título oficial
section	Capítulo, competência ou seção
text	Texto limpo usado na recuperação
sourceUrl	Endereço da publicação oficial
publishedAt	Data ou edição do documento
page	Página física ou lógica da evidência
checksum	Verificação da versão exata do arquivo

Substituição no protótipo

Para um primeiro piloto, posso substituir o conteúdo de **server/corpus.ts** por registros reais no mesmo formato. Para uma coleção maior, a melhor evolução é criar um processo de ingestão separado, salvar os chunks em armazenamento próprio e carregar o índice na inicialização. O endpoint de benchmark pode continuar igual.

Conjunto de perguntas e respostas

A qualidade da comparação depende mais do conjunto de avaliação do que do tamanho da interface. Eu prepararia perguntas com diferentes níveis de dificuldade e registraria, antes das execuções, a resposta esperada e os trechos considerados relevantes.

Tipo de pergunta	O que observa	Exemplo de intenção
Localização direta	Recuperação de uma afirmação explícita	Identificar uma definição
Síntese	Combinação de duas ou mais passagens	Relacionar competências
Comparação	Cobertura de diferenças documentadas	Distinguir igualdade e equidade
Sem resposta	Capacidade de recusar com segurança	Pergunta fora do corpus
Termo ambíguo	Robustez do ranking e da resposta	Palavra usada em mais de uma seção

Anotação

Cada pergunta deveria ter um identificador, uma categoria, uma resposta de referência e uma lista de chunks relevantes. Quando possível, duas pessoas fariam a anotação de forma independente e resolveriam divergências antes do benchmark. Isso reduz o risco de adaptar o gabarito ao comportamento observado do modelo.

Protocolo de execução

- Fixar versão do corpus, modelo, prompts, temperatura, top k e tamanho dos chunks.
- Executar as mesmas perguntas nos dois métodos e registrar erros e recusas, não apenas respostas bem-sucedidas.
- Repetir chamadas quando houver aleatoriedade e reportar média, dispersão e intervalo de confiança.
- Separar um conjunto usado para ajustar os parâmetros de outro usado para apresentar o resultado final.
- Guardar o resultado bruto antes de calcular os agregados.

Privacidade e direitos de uso

Documentos públicos também exigem controle de versão e atribuição. Se eu incluir avaliações, relatórios escolares ou material de estudantes, preciso remover dados pessoais e confirmar a base legal para tratamento e envio a provedores externos. Dados sensíveis não devem entrar no modo de API sem uma decisão institucional específica.

Como interpretar as métricas

As métricas atuais se dividem entre operação e qualidade. Tokens, latência e custo são observações diretas ou estimativas simples. Cobertura, aderência e fundamentação são heurísticas lexicais. Elas ajudam a detectar padrões, mas não substituem julgamento humano.

Métrica	Leitura correta	Limite principal
Latência	Tempo da chamada no servidor	Varia com rede, fila e cache
Tokens	Uso informado pela API ou estimado no demo	A estimativa local é aproximada
Custo	Aplicação de uma tabela de preço ao uso	Preços precisam ser atualizados
Cobertura	Sobreposição entre gabarito e resposta	Paráfrases podem receber nota baixa
Aderência	Termos da resposta presentes nas fontes	Copiar a fonte pode inflar o valor
Citações	Proporção de marcadores que apontam para chunks existentes	Não prova que a fonte sustenta a frase
Fundamentação	Proporção de afirmações com sobreposição suficiente	O limiar lexical é arbitrário

Avaliação recomendada

Eu usaria as métricas automáticas para triagem e acrescentaria uma avaliação humana cega. O avaliador não deveria saber qual método gerou cada resposta. A rubrica pode pontuar correção, completude, suporte nas fontes, clareza e comportamento diante de perguntas sem resposta.

Recuperação e geração precisam ser avaliadas separadamente

Uma resposta ruim pode decorrer de uma recuperação inadequada ou de uma geração que ignorou bons trechos. Por isso, o conjunto anotado deve permitir calcular medidas como recall dos chunks relevantes e precisão nas primeiras posições. Só depois faz sentido relacionar a qualidade da recuperação à qualidade da resposta.

Pontos de atenção

O protótipo foi construído para tornar a comparação observável. Algumas escolhas ainda são deliberadamente simples e devem ser tratadas como parte do desenho experimental.

O Direct não possui fonte de referência

Na implementação atual, a aderência do Direct é zero porque ele não recebe documentos. Isso descreve a ausência de rastreabilidade naquele caminho; não significa que toda resposta direta esteja factual ou semanticamente errada. A comparação de correção deve usar o gabarito e avaliação humana.

BM25 é um baseline, não a única forma de RAG

A busca lexical funciona bem quando a pergunta e a fonte compartilham vocabulário. Ela pode falhar com sinônimos e formulações muito diferentes. Embeddings ou busca híbrida são extensões naturais, mas devem ser comparados como novas condições experimentais.

O modo demo não mede desempenho de modelo

As respostas determinísticas foram escritas para exercitar o pipeline. Seus tokens são estimados pela quantidade de caracteres e sua latência é local. Qualquer tabela de resultado acadêmico deve excluir esse modo.

Custo e modelos mudam

A tabela de preços está embutida no servidor e pode ficar desatualizada. Também é possível que um identificador de modelo seja substituído pelo provedor. Antes de uma coleta, eu registraria preços, versão do modelo, data e região da chamada.

Persistência local tem limite operacional

O arquivo JSON é adequado ao protótipo, mas não oferece concorrência, controle de acesso ou auditoria completa. Um estudo com vários usuários deve usar banco de dados, registrar prompts e parâmetros integralmente e impedir alterações silenciosas nos resultados brutos.

As chamadas paralelas não eliminam variação externa

Executar Direct e RAG em paralelo reduz a diferença temporal entre eles, mas cada chamada ainda pode enfrentar filas e rotas distintas. A análise de latência precisa de várias repetições e deve apresentar distribuição, não apenas uma média.

Próximos passos que eu adotaria

Eu manteria a aplicação como base e evoluiria o experimento em etapas. A prioridade é melhorar a validade do estudo antes de adicionar mais recursos visuais.

Prioridade	Entrega	Critério de conclusão
1	Corpus oficial versionado	Fontes, páginas, URLs e checksum registrados
2	Dataset de avaliação	Perguntas, gabaritos e relevância revisados
3	Métricas de recuperação	Recall, precisão e ranking calculados
4	Avaliação humana cega	Rubrica definida e avaliadores sem identificação do método
5	Repetição estatística	Múltiplas execuções e intervalos de confiança
6	Baselines adicionais	BM25 comparado a vetorial e híbrido
7	Exportação	Resultados brutos e agregados em CSV ou JSON

Com essas etapas, o protótipo deixa de ser apenas uma demonstração de arquitetura e passa a sustentar uma comparação experimental defensável. A principal contribuição não será provar que RAG sempre vence. Será mostrar em quais tipos de pergunta a recuperação melhora a fidelidade, quanto custa e quais falhas ainda permanecem.

Referência rápida

- Desenvolvimento: `npm run dev`
- Testes: `npm test`
- Verificação de tipos: `npm run check`
- Build: `npm run build`
- Servidor de produção: `npm start`
- Histórico local: `data/runs.json`